

Audio-Foto-Pipeline Projektdokumentation

29. Mai 2026

1. Problembeschreibung

Bei Baubegehungen von Infrastrukturprojekte, Schadenserfassung, Baustellenprotokoll, erfasst ein Bauleiter Befunde vor Ort. Die manuelle Nachbearbeitung (Notizen abtippen, Fotos den richtigen Befunden zuordnen, Dokument erstellen) ist zeitaufwändig und fehleranfällig. In der Praxis führt dieser Aufwand dazu, dass Dokumentationen unvollständig bleiben oder gänzlich ausbleiben, Befunde geraten in Vergessenheit.

Ich selbst habe eine Ausbildung als Zeichner Fachrichtung Ingenieurbau absolviert und arbeite heute im Ingenieurbüro Emch+Berger WSB AG, langezeit als Zeichner, mittlerweile aber in der IT-Abteilung. Diese Kombination ermöglichte es, technische Anforderungen aus dem Baubereich direkt in eine praxisnahe Softwarelösung zu übersetzen.

2. Lösungsansatz

Der Lösungsansatz entstand aus direkten Gesprächen mit Projektleitern, die täglich auf Baustellen arbeiten. Ihre geschilderten Probleme bildeten die Grundlage für die entwickelte Lösung mit dem Ziel, dass sie tatsächlich im Unternehmen eingesetzt werden kann.

Ziel ist es, Befunde direkt vor Ort per Smartphone aufnehmen zu können, ohne Internetverbindung, ohne zusätzliche Software. Zurück im Büro übernimmt die Pipeline die Weiterverarbeitung vollständig automatisch: Transkription, Textbereinigung und Export in ein strukturiertes Word-Protokoll.

Die Pipeline eignet sich überall dort, wo Befunde, Zustände oder Beobachtungen dokumentiert werden müssen. Sie lässt sich mit oder ohne Fotos einsetzen. Wer nur Audiodiktate transkribieren möchte, kann den Foto-Schritt überspringen, wer nur eine Fotodokumentation benötigt, kann den Audio-Schritt weglassen, wer beides möchte, verknüpft Aufnahmen und Fotos automatisch zu einem vollständigen Bild-Text-Protokoll.

Die Pipeline besteht aus vier aufeinanderfolgenden Modulen:

Modul	Datei	Aufgabe
1	match.py	Liest EXIF-Zeitstempel der Fotos und Aufnahmezeitpunkt der Audiodateien (aus Dateiname oder mtime). Ordnet Fotos den Audio-Abschnitten chronologisch zu.
2	transcribe.py	Sendet Audiodateien an die OpenAI Whisper API, gibt strukturiertes Transkript mit Segment-Zeitstempeln zurück (max. 25 MB pro Datei).
3	format.py	Bereinigt das Transkript regelbasiert: Füllwörter entfernen, Segmente nach Sprachpausen gruppieren. Kein LLM, nur Textanalyse.
4	export.py	Erstellt ein Word-Dokument (.docx) mit Titelseite und eingebetteten Fotogalerien nach dem jeweiligen Audio-Abschnitt.

Eingabe der Inputdaten: Erfolgt in Ordner (nicht Einzeldateien), so können mehrere Aufnahmen und Fotos einer Begehung in einem Durchgang verarbeitet werden. Standardmässig wird auf die mitgelieferten Beispielordner *data/audio/* und *data/fotos/* verwiesen. Fotos werden anhand ihrer Zeitstempel automatisch korrekt in die Timeline eingeordnet.

3. Herausforderungen und Erfahrungen

Zeitstempel-Zuverlässigkeit: Zunächst wurde versucht, den Aufnahmezeitpunkt aus dem MP4-Tag der Audiodatei zu lesen. Dies erwies sich als unzuverlässig, da Dateisystem-Metadaten beim Übertragen überschrieben werden.

→ Lösung: ein Audio-Recorder, der den Zeitpunkt direkt im Dateinamen speichert (Format: YYYYMMDD-HHMMSS.m4a). Bei falschem Format fällt der Code auf die mtime zurück, mit Warnung.

API-Key-Handling: Aus Sicherheitsgründen wird der API-Key normalerweise in einer .env-Datei gespeichert und config.json bleibt leer, so gelangt der Key nicht ins Git-Repository. Für die Abgabe wurde er direkt in config.json eingetragen, damit der Dozent ohne zusätzliche Konfiguration starten kann. Der Code prüft zuerst config.json und nimmt .env als Fallback.

Code-Strukturierung: Der Code wurde mit AI-Unterstützung einheitlich formatiert. Alle Docstrings und Kommentare sind auf Deutsch. Die Word-Vorlage wurde AI-gestützt generiert, da kein Standardskript für python-docx existierte.

4. Handhabung und Ausführen des Projekts

```
python -m venv .venv
source .venv/bin/activate    # macOS/Linux   |   Windows: .venv\Scripts\activate
pip install -r requirements.txt
python main.py
```

Die Pipeline startet interaktiv und fragt Ordner, Projektname, Projektnummer und Aufnehmer ab. Bei den Ordnerabfragen stehen drei Eingabemöglichkeiten zur Verfügung:

- **Enter:** verwendet den Standardpfad (*data/audio/* bzw. *data/fotos/*)
- **Eigener Pfad:** beliebiger Ordner auf dem Dateisystem
- **Punkt (.):** überspringt diesen Schritt (z.B. nur Audio ohne Fotos)

Beispieldaten: Im Projektordner unter *data/audio/* und *data/fotos/* liegen reale Aufnahmen einer Zustandsaufnahme einer Quartierstrasse im Quartier Konstanz in Rothenburg. Der Dozent kann die Pipeline sofort starten, ein Drücken der Eingabetaste bei den Ordnerabfragen genügt, um die enthaltenen Beispieldaten zu verwenden.

Ausgabe: Die Pipeline erstellt im Ordner *output/* die Zwischendateien *mapping.json*, *transkript.json* und *befunde.json* sowie den fertigen Bericht *bericht.docx*. Ein Beispielprotokoll der Quartierstrassen-Begehung liegt dem Abgabeordner als *bericht.pdf* bei.

Tests: `python -m pytest tests/`

5. Verwendete Bibliotheken / Quellen

Claude AI (Anthropic) unterstützte bei Codeentwicklung, Code-Review, Strukturierung und Formatierung des Word-Outputs. Die eingesetzten Bibliotheken und Quellen sind hier nochmals aufgelistet:

Bibliothek	Verwendung	Quelle
openai	Whisper API (Speech-to-Text)	https://platform.openai.com/docs/guides/speech-to-text
python-docx	Word-Dokumente erstellen	https://python-docx.readthedocs.io
Pillow	EXIF-Daten lesen, HEIC konvertieren	https://pillow.readthedocs.io
pillow-heif	HEIC/HEIF-Unterstützung	https://pypi.org/project/pillow-heif/
mutagen	M4A-Metadaten lesen	https://mutagen.readthedocs.io